

Раздел 1. Цель работы

Цель работы - реализовать спроектированный на предыдущем этапе ETL-процесс на языке Python, выполнить очистку и преобразование данных и подготовить очищенный набор данных для загрузки в хранилище интеллектуальной системы предиктивного обслуживания.

Задачи работы:

1. реализовать извлечение данных из исходного источника;
2. выполнить все запланированные трансформации;
3. обработать пропуски, дубликаты и выбросы;
4. сохранить очищенные данные в промежуточный файл;
5. сформировать отчёт о выполненной очистке с метриками качества.

Исходными данными служит набор AI4I 2020 Predictive Maintenance Dataset (UCI Machine Learning Repository, идентификатор 601, лицензия CC BY 4.0), выбранный в практической работе № 1. Проект ETL-процесса - спецификация трансформаций, схема конвейера, стратегии обработки каждого поля - разработан в практической работе № 3 по результатам разведочного анализа из практической работы № 2. Настоящая работа реализует этот проект в виде воспроизводимой программы и экспериментально проверяет принятые в нём решения.

Раздел 2. Реализация ETL-процесса

2.1. Инструментарий и состав программы

Как и на предыдущих этапах, использовано только свободное программное обеспечение (таблица 1).

Таблица 1 - Использованное программное обеспечение и библиотеки

Наименование	Версия	Назначение	Лицензия
Python	3.11.15	Язык программирования, среда выполнения ETL	PSF License
pandas	3.0.6	Загрузка CSV, трансформации, группировки, сохранение	BSD-3-Clause

Наименование	Версия	Назначение	Лицензия
NumPy	2.4.6	Векторные вычисления производных признаков	BSD-3-Clause
PyArrow	25.0.1	Сохранение в колоночный формат Parquet	Apache-2.0
SciPy	1.17.1	Статистические характеристики (z-оценка)	BSD-3-Clause
Matplotlib	3.11.2	Визуализация результатов «до» и «после»	PSF-based

Программа разделена на модули по назначению (таблица 2). Конвейер запускается одной командой `python etl.py`; повторный запуск на тех же исходных данных даёт побайтно одинаковый результат, что обеспечивает воспроизводимость.

Таблица 2 - Состав программы

Файл	Назначение
etl.py	ETL-конвейер: функции extract, трансформации T1-T9, validate, load
test_robustness.py	Проверка устойчивости: внесение дефектов известного объёма и сравнение результата с эталоном; сравнение стратегий заполнения пропусков
figures.py	Построение рисунков отчёта
data/ai4i2020.csv	Исходный файл (сырые данные)
data/ data_cleaned.csv, .parquet	Очищенный набор данных - результат ETL
data/quality_report.csv	Отчёт о качестве: тип, число уникальных значений, пропуски, диапазон
data/etl_log.csv, etl_stats.json	Журнал операций и метрики выполнения

2.2. Схема ETL-процесса

Конвейер состоит из этапа извлечения, девяти шагов трансформации, валидации и сохранения (рисунок 1). Порядок шагов выбран так, чтобы каждый следующий работал с уже подготовленными данными: ошибочные значения сначала превращаются в пропуски (T3, T4), затем удаляются дубликаты (T5), и только после этого заполняются пропуски (T6) - иначе дубликаты исказили бы медианы, а ошибочные значения попали бы в заполнение.

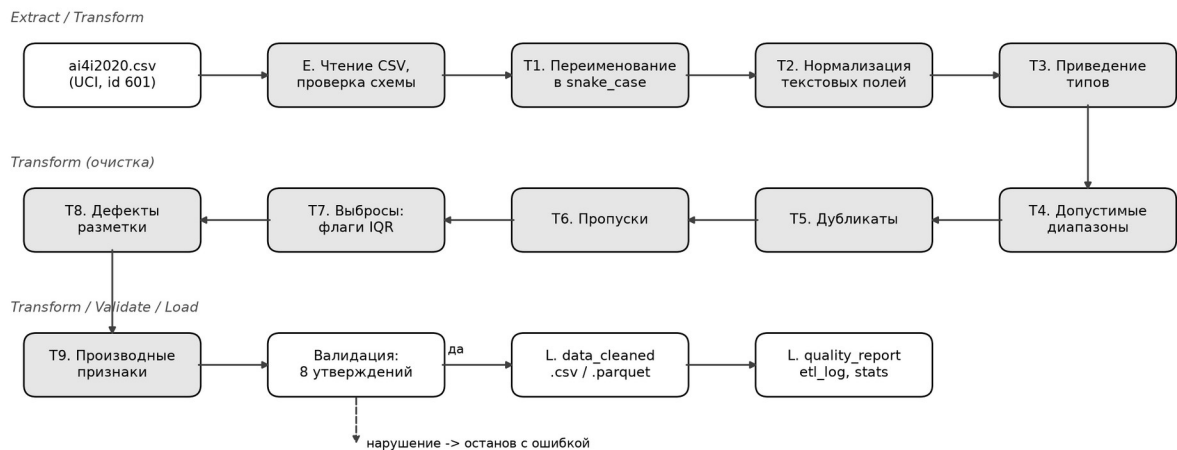


Рисунок 1 - Схема ETL-процесса

Если хотя бы одно утверждение валидации нарушено, конвейер завершается с ошибкой и файлы результата не перезаписываются. Это защищает хранилище от загрузки некорректных данных.

2.3. Код основных этапов

Извлечение. Файл читается в кодировке utf-8-sig (в файле есть BOM, см. отчёт по лабе 2), после чего проверяется наличие всех ожидаемых полей:

```
def extract(path=RAW):
    df = pd.read_csv(path, encoding='utf-8-sig')
    missing_cols = set(RENAME) - set(df.columns)
    if missing_cols:
        raise ValueError(f'В источнике нет полей: {sorted(missing_cols)}')
    return df
```

Нормализация текста и приведение типов. Лишние пробелы и регистр исправляются; класс качества, если он отсутствует или ошибочен, восстанавливается по первой букве Product ID - в лабе 2 показано, что они совпадают во всех записях. Нечисловые значения в числовых полях превращаются в пропуск, класс качества - в упорядоченную категорию $L < M < H$:

```
df['product_id'] = df['product_id'].astype(str).str.strip().str.upper()
df['quality_class'] = df['quality_class'].astype(str).str.strip().str.upper()
bad = ~df['quality_class'].isin(CLASSES)
restored = bad & df['product_id'].str[0].isin(CLASSES)
df.loc[restored, 'quality_class'] = df.loc[restored, 'product_id'].str[0]
...
for c in SENSORS + LABELS + ['udi']:
    df[c] = pd.to_numeric(df[c], errors='coerce')
df['quality_class'] = pd.Categorical(df['quality_class'],
                                    categories=['L', 'M', 'H'], ordered=True)
```

Проверка допустимых диапазонов. Значения вне физически возможного диапазона (например, температура 27 К - типичная ошибка записи градусов Цельсия вместо кельвинов) считаются ошибкой датчика и заменяются пропуском, который затем заполняется:

```
VALID_RANGE = {'air_temp_k': (273, 343), 'process_temp_k': (273, 373),
               'rot_speed_rpm': (1, 5000), 'torque_nm': (0, 150),
               'tool_wear_min': (0, 400)}
for c, (lo, hi) in VALID_RANGE.items():
    bad = df[c].notna() & ~df[c].between(lo, hi)
    df.loc[bad, c] = np.nan
inv = df['process_temp_k'] < df['air_temp_k'] # процесс не холоднее воздуха
df.loc[inv, 'process_temp_k'] = np.nan
```

Дубликаты удаляются в два прохода - полные совпадения строк, затем повторы ключа UDI (сохраняется первая запись):

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset='udi', keep='first')
```

Пропуски. Стратегия задана отдельно для каждого признака (лаба 3, п. 2.2) и проверена на искусственно скрытых значениях (раздел 3.2). Записи без ключа или без обобщённой метки отказа не восстанавливаются: метка служит эталоном для проверки базы правил:

```
key_bad = df['udi'].isna() | df['product_id'].isna() | df['machine_failure'].isna()
sens_bad = df[SENSORS].isna().sum(axis=1) > len(SENSORS) // 2
df = df[~(key_bad | sens_bad)].copy()
# температуры - плавный временной ряд: линейная интерполяция
for c in ('air_temp_k', 'process_temp_k'):
    df[c] = df[c].interpolate(limit_direction='both').round(1)
# момент - через почти постоянную мощность привода
df.loc[m_t & ~m_n, 'torque_nm'] = POWER_MED / (df['rot_speed_rpm'] * 2*np.pi/60)
# остальное - медиана внутри класса качества
med = df.groupby('quality_class', observed=True)[c].transform('median')
df.loc[df[c].isna(), c] = med
```

Износ инструмента восстанавливается по нормативному шагу: внутри цикла инструмента износ растёт на 2, 3 или 5 минут в зависимости от класса предыдущего изделия (L, M, H). Это правило выполняется на всех 9 880 шагах набора без исключений, поэтому пропуск вычисляется точнее, чем интерполяцией:

```
for i in miss:
    fwd = w[i - 1] + step[i - 1] # предыдущее значение + шаг
    nxt = следующее известное значение
    if nxt is not None and w[nxt] < w[i - 1]: # между ними была замена инструмента
        w[i] = max(w[nxt] - step[i:nxt].sum(), 0)
    else:
        w[i] = fwd
```

Выбросы не удаляются, а помечаются флагами. Границы межквартильного размаха зафиксированы по эталонному набору (лаба 2, таблица 4), чтобы новые партии данных оценивались по тем же порогам:

```
IQR_BOUNDS = {'rot_speed_rpm': (1139.5, 1895.5), 'torque_nm': (12.8, 67.2)}
for c, flag in [('rot_speed_rpm', 'is_outlier_speed'),
                ('torque_nm', 'is_outlier_torque')]:
    lo, hi = IQR_BOUNDS[c]
    df[f'flag'] = ((df[c] <= lo) | (df[c] >= hi)).astype('int8')
```

Дефекты разметки и производные признаки:

```
sub = df[['twf', 'hdf', 'pwf', 'osf']].sum(axis=1)
df['failure_type_unknown'] = ((df['machine_failure'] == 1) & (sub == 0)).astype('int8')
df = df.rename(columns={'rnf': 'rnf_ref'}) # RNF - справочный признак
df['temp_diff_k'] = df['process_temp_k'] - df['air_temp_k']
df['power_w'] = df['torque_nm'] * df['rot_speed_rpm'] * 2 * np.pi / 60
df['wear_torque'] = df['tool_wear_min'] * df['torque_nm']
df['osf_limit'] = df['quality_class'].map({'L': 11000, 'M': 12000, 'H': 13000})
df['osf_margin'] = df['osf_limit'] - df['wear_torque']
df['power_margin'] = np.minimum(df['power_w'] - 3500, 9000 - df['power_w'])
df['tool_cycle_id'] = (df['tool_wear_min'].diff() < 0).cumsum() + 1
```

Валидация и сохранение. Результат проверяется набором утверждений; при успехе данные сохраняются в CSV и Parquet, формируются отчёт о качестве и журнал операций:

```
checks = {
    'нет пропусков': df.isnull().sum().sum() == 0,
    'UDI уникален': df['udi'].is_unique,
    'класс совпадает с буквой Product ID': ...,
    'показания в допустимых диапазонах': ...,
    'machine_failure = OR(подтипов) или failure_type_unknown': ...,
    'объём не меньше 500 записей': len(df) >= 500, ...}
df.to_csv('data/data_cleaned.csv', index=False)
df.to_parquet('data/data_cleaned.parquet', index=False)
quality_report(df).to_csv('data/quality_report.csv', index=False)
```

Полный текст программы приведён в приложении А.

2.4. Выполненные трансформации

Соответствие реализованных шагов проекту из лабы 3 и число затронутых записей на исходном наборе приведены в таблице 3.

Таблица 3 - Выполненные трансформации

Шаг	Операция и метод pandas	Основание	Затронуты
T1	Переименование полей в snake_case: rename	Лаба 3, п. 5.1	14 полей
T2	Нормализация текста: str.strip, str.upper; восстановление класса по Product ID	Лаба 3, п. 5.3-5.4	0
T3	Приведение типов: to_numeric(errors='coerce'), Categorical(ordered=True), понижение разрядности (int8, int16, int32)	Лаба 3, табл. 3	28 полей

Шаг	Операция и метод pandas	Основание	Затронуто
T4	Проверка физически допустимых диапазонов: <code>between</code>	Лаба 3, п. 5.5-5.9	0
T5	Удаление дубликатов: <code>drop_duplicates()</code> , <code>drop_duplicates(subset='udi')</code>	Лаба 3, п. 5.2	0
T6	Обработка пропусков: <code>dropna</code> -логика, <code>interpolate</code> , <code>groupby().transform('median')</code>	Лаба 3, п. 2.2	0
T7	Флаги выбросов по IQR: <code>is_outlier_speed</code> , <code>is_outlier_torque</code>	Лаба 3, п. 5.7-5.8	418 и 69
T8	Флаг <code>failure_type_unknown</code> ; <code>failure_mode</code> по приоритету подтипов; <code>RNF -> rnf_ref</code>	Лаба 3, п. 5.10-5.12	9, 23, 18
T9	Производные признаки: <code>temp_diff_k</code> , <code>power_w</code> , <code>power_margin</code> , <code>wear_torque</code> , <code>osf_limit</code> , <code>osf_margin</code> , <code>tool_cycle_id</code> , <code>cycle_pos</code>	Лаба 3, п. 5.13-5.14	8 полей
V	Валидация: 8 утверждений	Лаба 3, табл. 6	все пройдены
L	Сохранение: <code>to_csv</code> , <code>to_parquet</code>	Лаба 3, п. 4.1	10 000 записей

Все шаги проекта реализованы без изменения состава и порядка; параметры конвейера (диапазоны, границы IQR, пороги, приоритет подтипов) совпадают со справочными параметрами из лабы 3 (таблица 5). Шаг T4 дополнительно реализует нефункциональное требование № 4 лабы 1 о корректной обработке ошибочных входных данных.

Раздел 3. Статистика очистки

Исходный набор содержит 10 000 записей и 14 полей; после обработки - 10 000 записей и 28 полей. Ни одна запись не удалена. Конвейер выполняется за 0,25 с, что многократно лучше требования лабы 1 (пакетная обработка 10 000 записей - не более 60 с, таблица 9, п. 1).

3.1. Дубликаты

На исходном наборе удалено 0 записей: полных дубликатов нет, значения UDI и Product ID уникальны. Результат совпадает с выводом лабы 2 (п. 4.2). Шаг сохранён в конвейере, поскольку при загрузке новых партий телеметрии повторная передача пакета - типичная ошибка интеграции со SCADA.

3.2. Пропуски

На исходном наборе обработано 0 пропусков - доля заполненности 100 % по всем полям (лаба 2, п. 4.1). Стратегии заполнения для будущих партий выбраны в лабе 3 (п. 2.2) по структуре признаков: автокорреляции соседних записей и физическим связям. Здесь этот выбор проверен экспериментально: на эталонном наборе скрывалось 2 % значений каждого признака, они восстанавливались разными способами, и рассчитывалась средняя абсолютная ошибка (MAE) относительно истинных значений. Результаты приведены в таблице 4 и на рисунке 2.

Таблица 4 - Средняя абсолютная ошибка восстановления пропусков разными стратегиями (выбранная стратегия выделена)

Стратегия	Т воздуха, К	Т процесса, К	Обороты, об/мин	Момент, Н·м	Износ, мин
Среднее	1,69	1,20	133,69	7,46	55,69
Медиана	1,68	1,20	131,22	7,47	55,67
Медиана по классу	1,68	1,19	131,40	7,46	55,52
Интерполяция	0,04	0,04	-	-	4,22
По связи признаков	0,92	0,86	245,76	5,01	-
Шаг износа по классу	-	-	-	-	3,34
СКО признака	2,00	1,48	179,28	9,97	63,65

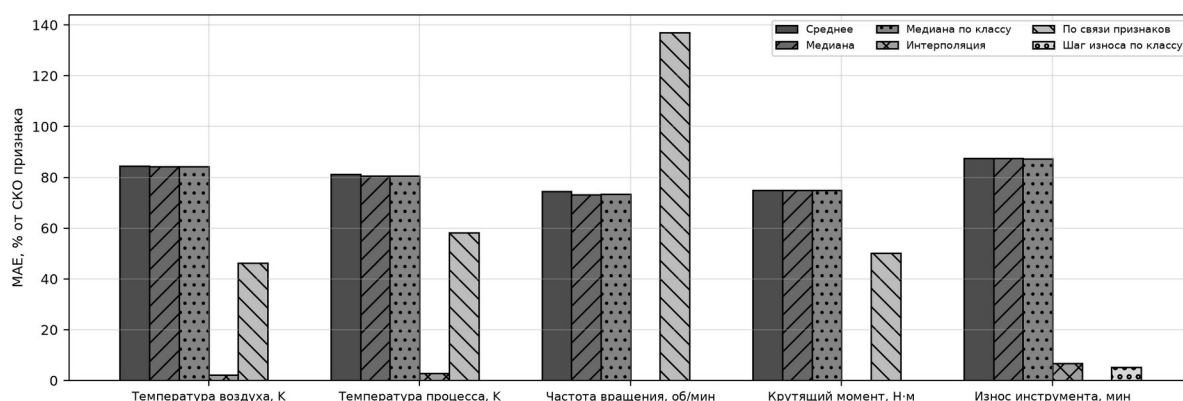


Рисунок 2 - Ошибка восстановления пропусков в процентах от СКО признака

Эксперимент подтвердил решения лабы 3:

- стандартные стратегии из методических указаний (среднее, медиана) дают ошибку 73-88 % от СКО признака, то есть почти не лучше случайного значения из распределения. Расчёт медианы внутри

класса качества точность не повышает: показания датчиков от класса изделия практически не зависят;

- температуры воздуха и процесса в наборе изменяются плавно от записи к записи (они сгенерированы как случайное блуждание), поэтому линейная интерполяция по соседним записям снижает ошибку в 40 раз - до 0,04 К. Это частный случай рекомендации методических указаний применять интерполяцию к плавным временным рядам;
- крутящий момент восстанавливается через почти постоянную мощность привода (лаба 2, п. 4.4, коэффициент корреляции с оборотами -0,875): $M = P / \omega$ при медианной мощности 6 271 Вт. Ошибка снижается на треть по сравнению с медианой;
- обратный пересчёт оборотов по моменту, напротив, вдвое хуже медианы: при малом моменте деление даёт большую ошибку. Для оборотов выбрана медиана по классу качества, устойчивая к выбросам (асимметрия признака - 1,99);
- износ инструмента восстанавливается по нормативному шагу 2/3/5 мин с ошибкой 3,3 мин (5 % от СКО) - точнее интерполяции, которая ошибается в точках замены инструмента;
- класс качества восстанавливается по букве Product ID, а при её отсутствии - модой; подтипы отказа - нулём с последующей проверкой согласованности (шаг T8).

3.3. Выбросы

Выбросы определены методом межквартильного размаха с границами из лабы 2 и дополнительно методом z-оценки ($|z| > 3$) для сравнения (таблица 5).

Таблица 5 - Обработка выбросов

Признак	Границы IQR	Выбросов (IQR)	Из них отказов	Выбросов ($ z > 3$)	Решение
Частота вращения, об/мин	1139,5 ... 1895,5	418	35 (8,4 %)	164	Сохранить, флаг is_outlier_speed
Крутящий момент, Н·м	12,8 ... 67,2	69	62 (89,9 %)	25	Сохранить, флаг is_outlier_torque
Остальные датчики	по лабе 2	0	-	0	Обработка не требуется

Из четырёх стратегий методических указаний (удаление, винзоризация, логарифмирование, сохранение) выбрано сохранение с флагом. Доля отказов среди записей с аномальным моментом - 89,9 % против 3,39 % в среднем, то есть выбросы - это предотказные режимы, а не ошибки измерения. Удаление уничтожило бы сигнал, винзоризация исказила бы значения, по которым срабатывают правила PWF и OSF, а логарифмирование не нужно производственным правилам, работающим с исходными физическими величинами. Метод z-оценки находит меньше выбросов (164 и 25), поскольку предполагает нормальное распределение, а частота вращения ему не подчиняется. Поэтому основным оставлен метод IQR.

Пять значений момента лежат точно на границах 12,8 и 67,2 Н·м, четыре из них - отказы. Граница включается в область выбросов, что согласуется с подсчётом лабы 2 (69 записей).

3.4. Дефекты разметки

Дефекты, выявленные в лабе 2 (п. 4.6), обработаны без удаления записей (таблица 6).

Таблица 6 - Обработка дефектов разметки

Дефект	Записей	Обработка
Отказ без указания подтипа	9	Флаг failure_type_unknown = 1, failure_mode = UNKNOWN; исключаются из обучения классификатора типа отказа
Подтип RNF при нулевой обобщённой метке	18	Поле RNF переименовано в rnf_ref и выведено из целевых переменных

Дефект	Записей	Обработка
Несколько подтипов в одной записи	23	Сохранены все бинарные метки и число подтипов (n_failure_modes); для одноклассовой постановки - failure_mode по приоритету PWF > OSF > HDF > TWF

Приоритет подтипов выбран по тяжести последствий: отказ по мощности и перегрузка повреждают привод и механику станка, нарушение теплоотвода требует проверки системы охлаждения, а отказ по износу устраняется заменой инструмента. Распределение итоговой метки failure_mode: NONE - 9 661, HDF - 106, PWF - 95, OSF - 86, TWF - 43, UNKNOWN - 9; сумма отказов - 339, что совпадает с числом записей Machine failure = 1. После выведения RNF обобщённая метка строго равна логическому ИЛИ четырёх подтипов во всех записях, кроме девяти помеченных как UNKNOWN.

3.5. Проверка устойчивости конвейера

Исходный набор чист, поэтому на нём ветви очистки не срабатывают. Чтобы проверить их, в копию набора внесены дефекты известного объёма (программа test_robustness.py, фиксированное зерно генератора). Результаты приведены в таблице 7.

Таблица 7 - Результаты проверки на данных с внесёнными дефектами

Внесённый дефект	Внесено	Результат
Пропуски датчиков (2 % каждого)	1 000	1 025 значений заполнено (включая 25 ошибочных из строк ниже)
Температура в °C вместо K	10	20 значений -> пропуск -> заполнено
Отрицательный момент	10	учтено в строке выше
Нечисловые значения («н/д»)	5	5 -> пропуск -> заполнено
Пробелы и нижний регистр в классе	30	41 значений нормализовано
Пустой класс качества	10	10 восстановлено по Product ID
Полные дубликаты	50	50 удалено
Повтор UDI с другими значениями	20	20 удалено
Записи с утратой 3 из 5 датчиков	3	3 удалено

Из 10 073 записей с дефектами конвейер вернул ровно 10 000 - исходный объём; все восемь утверждений валидации выполнены, доля

отказов не изменилась (3,39 %). Ошибка восстановления составила: температура воздуха - 0,03 К, температура процесса - 0,04 К, частота вращения - 126 об/мин, момент - 4,4 Н·м, износ - 1,0 мин. Тем самым нефункциональное требование № 4 лабы 1 (корректная обработка пропусков и выбросов без аварийного завершения) выполнено.

Раздел 4. Сравнение «до» и «после»

4.1. Сравнение статистик

Сравнение описательных статистик показаний датчиков до и после обработки исходного набора приведено в таблице 8.

Таблица 8 - Статистики показаний датчиков до / после обработки

Признак	Среднее	СКО	Минимум	Максимум	Значения
Т воздуха, К	300,00 / 300,00	2,00 / 2,00	295,3 / 295,3	304,5 / 304,5	совпадают
Т процесса, К	310,01 / 310,01	1,48 / 1,48	305,7 / 305,7	313,8 / 313,8	совпадают
Обороты, об/мин	1 538,78 / 1 538,78	179,28 / 179,28	1 168,0 / 1 168,0	2 886,0 / 2 886,0	совпадают
Момент, Н·м	39,99 / 39,99	9,97 / 9,97	3,8 / 3,8	76,6 / 76,6	совпадают
Износ, мин	107,95 / 107,95	63,65 / 63,65	0,0 / 0,0	253,0 / 253,0	совпадают

Значения всех пяти показаний совпадают с исходными поэлементно. Это ожидаемый и правильный результат: в исходных данных нет ни пропусков, ни ошибочных значений, а выбросы сохранены. Изменилась структура набора (таблица 9).

Таблица 9 - Изменение структуры набора данных

Показатель	До	После
Записей	10 000	10 000
Полей	14	28
Из них предикторов датчиков	5	5
Производных признаков	0	8
Служебных флагов	0	3 (выбросы - 2, неизвестный подтип - 1)
Целевых меток	6 (с RNF)	5 + failure_mode; RNF - справочный
Имена полей	с пробелами и скобками	snake_case
Тип класса качества	строка	упорядоченная категория L < M < H
Объём в памяти, КБ	1 162	1 105

Показатель	До	После
Размер файла CSV / Parquet, КБ	510 / -	1 121 / 469

Несмотря на увеличение числа полей вдвое, объём в памяти не вырос: метки и флаги хранятся как int8 вместо int64, класс качества - как категория. Формат Parquet занимает в 2,4 раза меньше места, чем CSV, и сохраняет типы данных, поэтому рекомендуется для последующих этапов.

4.2. Визуальное сравнение

Поскольку на исходном наборе значения не изменились, графики «до» и «после» для него совпали бы. Содержательное сравнение выполнено на данных с внесёнными дефектами (п. 3.5). Диаграммы размаха до и после очистки приведены на рисунке 3: значения 27 К в температуре воздуха и -5 Н·м в моменте до очистки видны как отдельные точки, после - отсутствуют, а выбросы частоты вращения и момента сохранены.

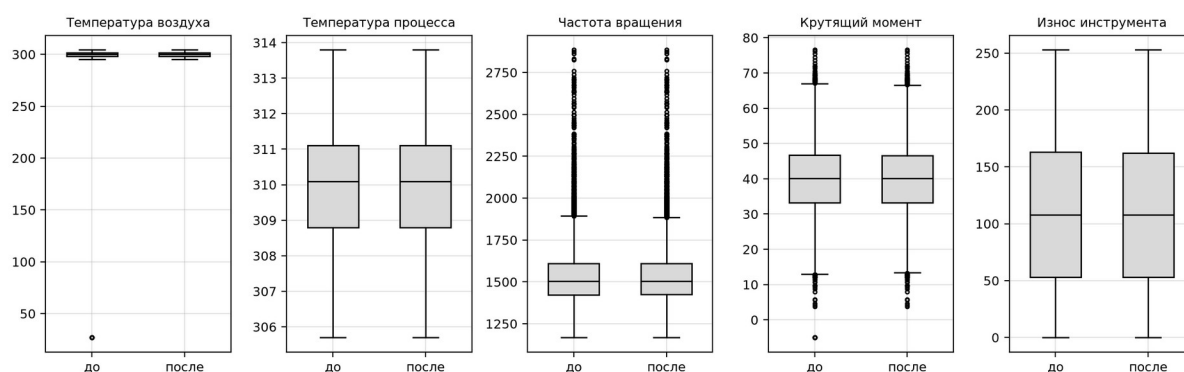


Рисунок 3 - Диаграммы размаха показаний датчиков до и после очистки (данные с внесёнными дефектами)

На рисунке 4 гистограммы исходных данных сопоставлены с результатом очистки данных с дефектами. Распределения практически совпадают: заполнение пропусков не создало искусственных пиков, которые неизбежно появились бы при заполнении средним или медианой.

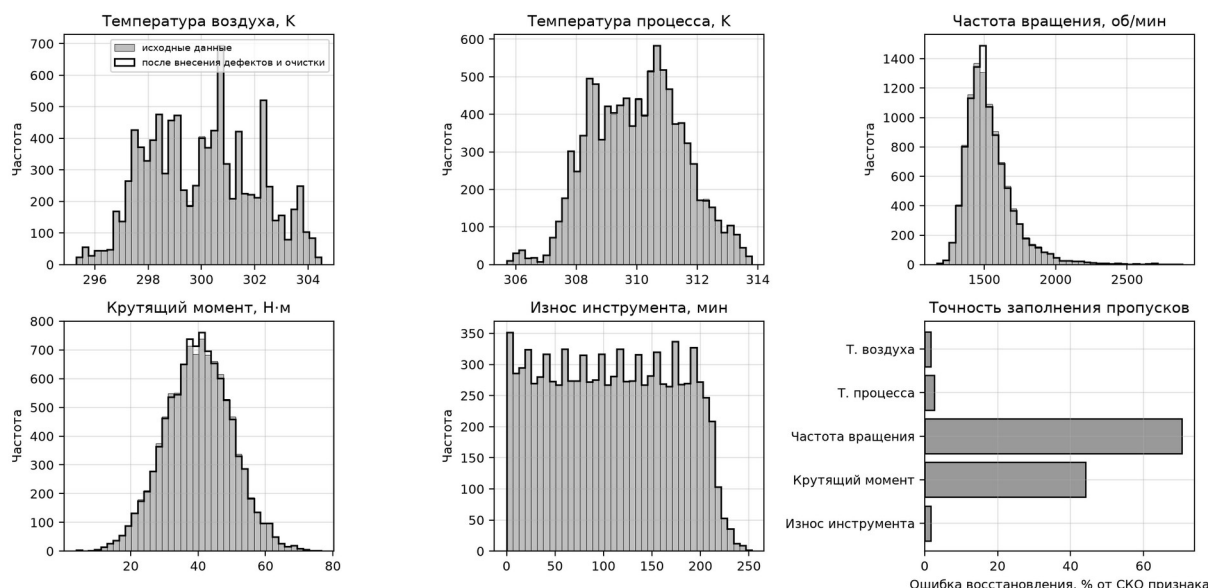


Рисунок 4 - Распределения исходных данных и данных после внесения дефектов и очистки

4.3. Новые признаки

Состав созданных признаков и их назначение в системе приведены в таблице 10.

Таблица 10 - Производные признаки

Признак	Формула	Назначение
temp_diff_k	$T_{\text{проц}} - T_{\text{возд}}$	Условие правила HDF ($< 8,6 \text{ K}$)
power_w	$M \cdot n \cdot 2\pi / 60$	Условие правила PWF
power_margin	$\min(P - 3500; 9000 - P)$	Запас до границы PWF; < 0 - вне диапазона; составляющая оценки риска для генетического алгоритма
wear_torque	$\text{износ} \cdot M$	Условие правила OSF
osf_limit	11 000 / 12 000 / 13 000 по классу	Слот фрейма-прототипа, наследуемый по классу качества
osf_margin	$\text{osf_limit} - \text{wear_torque}$	Запас до перегрузки; < 0 - условие OSF выполнено
tool_cycle_id	номер цикла: +1 при сбросе износа	Ключ экземпляра инструмента; журнал замен во фрейме
cycle_pos	номер изделия в цикле	Наработка инструмента в изделиях

Распределения трёх ключевых признаков для исправного состояния и для отказов приведены на рисунке 5. Пунктиром показаны пороги производственных правил: отказы сосредоточены именно за ними - при разности температур менее $8,6 \text{ K}$, при мощности вне диапазона $3500\text{--}9000 \text{ Вт}$ и при отрицательном запасе до порога перегрузки.

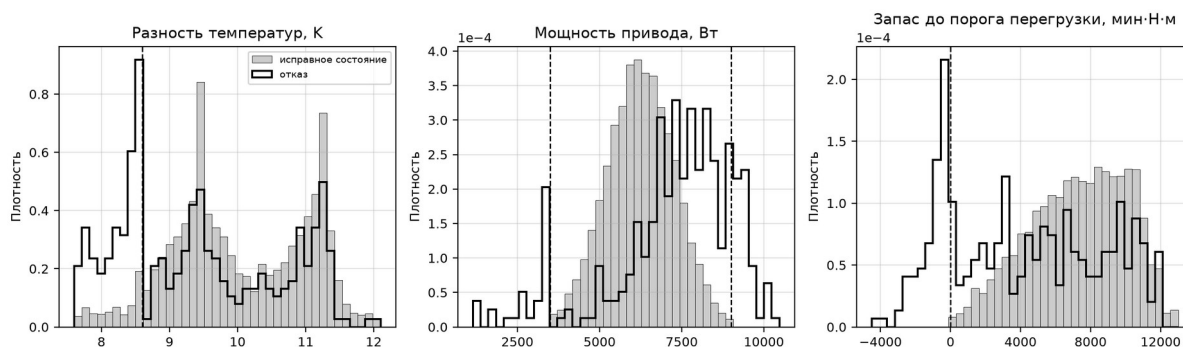


Рисунок 5 - Распределения производных признаков с порогами правил

Признак цикла инструмента предусмотрен проектом (лаба 3, п. 5.14). Износ инструмента сбрасывается в ноль 119 раз, то есть набор описывает 120 циклов эксплуатации инструмента со средней длиной 83,3 изделия (рисунок 6). Все 46 отказов TWF приходятся на последнюю запись цикла: отказ по износу - это событие замены инструмента. Износ в момент замены после отказа (198-253 мин, медиана 214) не отличается от износа при плановой замене (198-246 мин, медиана 215). Это объясняет низкую точность правила TWF, установленную в лабе 2 (5,4 %): по одному износу отличить отказ от плановой замены нельзя. Для генетического алгоритма это означает, что замену инструмента следует планировать до достижения 200 мин, а не реагировать на отказ.

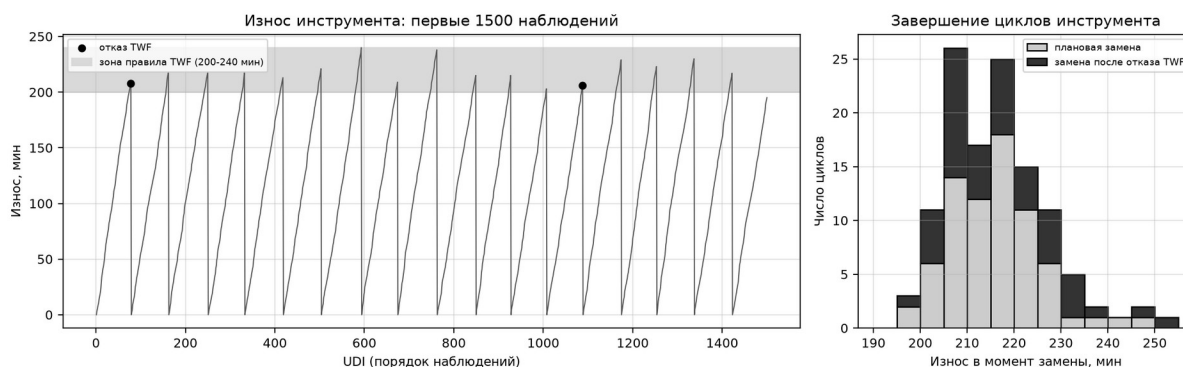


Рисунок 6 - Циклы эксплуатации инструмента

Раздел 5. Выводы о качестве полученных данных

В ходе работы проект ETL-процесса из лабы 3 реализован в виде ETL-конвейера на Python из этапа извлечения, девяти шагов трансформации, валидации и сохранения. Результат - набор `data_cleaned` из 10 000 записей и

28 полей в форматах CSV и Parquet, отчёт о качестве quality_report.csv и журнал операций etl_log.csv.

Качество полученных данных оценено как высокое и пригодное для загрузки в хранилище:

- полнота - 100 %: пропусков нет ни в одном из 28 полей;
- уникальность - ключи UDI и Product ID уникальны, дубликатов нет;
- корректность - все показания в физически допустимых диапазонах, класс качества согласован с Product ID, шаг износа соответствует нормативу во всех 9 880 случаях;
- согласованность разметки - после выведения RNF обобщённая метка равна логическому ИЛИ подтипов во всех записях, кроме девяти помеченных записей с неизвестным подтипом;
- информативность - выбросы сохранены и помечены флагами, поскольку 89,9 % записей с аномальным моментом соответствуют отказам.

Классические операции очистки на исходном наборе не изменили ни одного значения, поскольку данные синтетические и не содержат дефектов. Поэтому работоспособность конвейера проверена на копии с 1 138 внесёнными дефектами девяти видов: конвейер вернул исходный объём данных и прошёл все проверки. Стратегии заполнения пропусков, выбранные в лабе 3, подтверждены сравнением: интерполяция для температур, расчёт через мощность привода для момента, нормативный шаг для износа и медиана по классу для частоты вращения. Для температур, момента и износа они в 1,5-40 раз точнее заполнения средним.

Созданы восемь производных признаков, которые напрямую используются в условиях продукционных правил (temp_diff_k, power_w, wear_torque, osf_limit), в слотах фреймов (osf_limit, tool_cycle_id) и в оценке риска для целевой функции генетического алгоритма (power_margin, osf_margin). Выявлена структура циклов эксплуатации инструмента: 120 циклов, все отказы TWF совпадают с заменой инструмента. Это даёт

естественную сущность «Инструмент» для хранилища данных и объясняет вероятностную природу правила TWF.

Объём очищенных данных (10 000 записей при минимуме 500) достаточен для построения хранилища. Данные готовы к проектированию его схемы на следующем этапе: в них выделяются факты (показания датчиков и события отказа) и измерения (изделие, класс качества, цикл инструмента, тип отказа).

Список использованных источников

1. Маккинни У. Python и анализ данных: первичная обработка данных с применением pandas, NumPy и Jupyter / У. Маккинни ; пер. с англ. - 3-е изд. - Москва : ДМК Пресс, 2023. - 536 с. - ISBN 978-5-93700-174-0.
2. Очистка данных: проблемы и современные подходы // Хабр. - URL: <https://habr.com/ru/articles/548220/> (дата обращения: 05.10.2026).
3. Проблемы, методы и вызовы комплексной очистки данных // Хабр. - URL: <https://habr.com/ru/articles/548356/> (дата обращения: 05.10.2026).
4. Основы очистки данных в data science // Хабр. - URL: <https://habr.com/ru/articles/875662/> (дата обращения: 05.10.2026).
5. Очистка данных перед загрузкой в хранилище. Подробное руководство с техническими деталями // Хабр. - URL: <https://habr.com/ru/articles/794036/> (дата обращения: 05.10.2026).
6. Matzka S. Explainable Artificial Intelligence for Predictive Maintenance Applications / S. Matzka // Third International Conference on Artificial Intelligence for Industries (AI4I). - 2020. - P. 69-74.
7. AI4I 2020 Predictive Maintenance Dataset : [набор данных] // UCI Machine Learning Repository. - URL: <https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset> (дата обращения: 05.10.2026).

8. pandas documentation : [официальная документация библиотеки]. -
URL: <https://pandas.pydata.org/docs/> (дата обращения: 05.10.2026).

Приложение А. Листинг программы ETL-процесса

Ниже приведён полный текст модуля etl.py.

```
"""ETL-процесс подготовки данных AI4I 2020 (лабораторная работа 4, вариант 10).

Реализует проект ETL-процесса из лабы 3 (спецификация трансформаций, табл. 3),
составленный по результатам EDA из лабы 2:
извлечение -> нормализация текста -> приведение типов -> проверка допустимых
диапазонов -> дубликаты -> пропуски -> выбросы (флаги) -> дефекты разметки ->
производные признаки -> валидация -> сохранение (CSV, Parquet, отчёт о качестве).

Запуск: python etl.py (data/ai4i2020.csv -> data/data_cleaned.*)
"""
import hashlib
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd

HERE = Path(__file__).resolve().parent
RAW = HERE / 'data' / 'ai4i2020.csv'
OUT = HERE / 'data'

# --- справочные параметры конвейера -----
RENAME = {
    'UDI': 'udi',
    'Product ID': 'product_id',
    'Type': 'quality_class',
    'Air temperature [K]': 'air_temp_k',
    'Process temperature [K]': 'process_temp_k',
    'Rotational speed [rpm]': 'rot_speed_rpm',
    'Torque [Nm]': 'torque_nm',
    'Tool wear [min]': 'tool_wear_min',
    'Machine failure': 'machine_failure',
    'TWf': 'twf', 'HDF': 'hdf', 'PWF': 'pwf', 'OSF': 'osf', 'RNF': 'rnf',
}
SENSORS = ['air_temp_k', 'process_temp_k', 'rot_speed_rpm', 'torque_nm', 'tool_wear_min']
MODES = ['twf', 'hdf', 'pwf', 'osf'] # подтипы, входящие в целевую переменную
LABELS = ['machine_failure'] + MODES + ['rnf']
CLASSES = ['L', 'M', 'H']

# Физически допустимые диапазоны (паспорт станка, ISO 17359): значения вне
# диапазона считаются ошибкой датчика и заменяются пропуском.
VALID_RANGE = {
    'air_temp_k': (273.0, 343.0), # 0 ... 70 °C
    'process_temp_k': (273.0, 373.0), # 0 ... 100 °C
    'rot_speed_rpm': (1.0, 5000.0),
    'torque_nm': (0.0, 150.0),
    'tool_wear_min': (0.0, 400.0),
}

# Пороги правил из лабы 1 (табл. 4) и лабы 2 (табл. 7)
OSF_LIMIT = {'L': 11000, 'M': 12000, 'H': 13000}
WEAR_STEP = {'L': 2, 'M': 3, 'H': 5} # приращение износа за изделие
POWER_MIN, POWER_MAX = 3500.0, 9000.0
# Медианы связей между признаками (лаба 2, раздел 4.4): мощность привода
# используется для восстановления момента, разность температур - только
# в сравнении стратегий (test_robustness.py)
TEMP_DIFF_MED = 10.0 # K
POWER_MED = 6271.0 # Вт
# Границы IQR, рассчитанные на эталонном наборе (лаба 2, табл. 4). Фиксируются,
# чтобы новые партии данных оценивались по тем же порогам и заполненные
# значения не сдвигали границы.
IQR_BOUNDS = {'rot_speed_rpm': (1139.5, 1895.5), 'torque_nm': (12.8, 67.2)}
# Приоритет подтипов по тяжести последствий (для одноклассовой постановки)
PRIORITY = ['pwf', 'osf', 'hdf', 'twf']

def md5(path):
    return hashlib.md5(Path(path).read_bytes()).hexdigest()

# --- E: извлечение -----
def extract(path=RAW):
    df = pd.read_csv(path, encoding='utf-8-sig')
```

```

missing_cols = set(RENAME) - set(df.columns)
if missing_cols:
    raise ValueError(f'В источнике нет полей: {sorted(missing_cols)}')
return df

# --- T: трансформации -----
def rename(df, log):
    df = df.rename(columns=RENAME)[list(RENAME.values())]
    log.append(('Переименование полей в snake_case', len(RENAME), 'все поля'))
    return df

def normalize_text(df, log):
    before_pid = df['product_id'].astype(str)
    before_cls = df['quality_class'].astype(str)
    df['product_id'] = before_pid.str.strip().str.upper()
    df['quality_class'] = before_cls.str.strip().str.upper()
    changed = int((before_pid != df['product_id']).sum() + (before_cls !=
df['quality_class']).sum())
    # класс качества, отсутствующий или недопустимый, восстанавливается по букве Product ID
    bad = ~df['quality_class'].isin(CLASSES)
    restored = bad & df['product_id'].str[0].isin(CLASSES)
    df.loc[restored, 'quality_class'] = df.loc[restored, 'product_id'].str[0]
    df.loc[bad & ~restored, 'quality_class'] = np.nan
    mismatch = int((df['product_id'].str[0] != df['quality_class']).sum())
    df['product_serial'] = pd.to_numeric(df['product_id'].str[1:],
errors='coerce').astype('Int32')
    log.append(('Нормализация текстовых полей (пробелы, регистр)', changed, 'product_id,
quality_class'))
    log.append(('Восстановление класса качества по Product ID', int(restored.sum()),
'quality_class'))
    log.append(('Несоответствие буквы Product ID классу качества', mismatch, 'контроль'))
    return df

def cast_types(df, log):
    n_bad = 0
    for c in SENSORS + LABELS + ['udi']:
        conv = pd.to_numeric(df[c], errors='coerce')
        n_bad += int((conv.isna() & df[c].notna()).sum())
        df[c] = conv
    df['quality_class'] = pd.Categorical(df['quality_class'], categories=CLASSES,
ordered=True)
    log.append(('Приведение типов; нечисловые значения -> пропуск', n_bad, 'числовые поля'))
    return df

def check_ranges(df, log):
    total = 0
    for c, (lo, hi) in VALID_RANGE.items():
        bad = df[c].notna() & ~df[c].between(lo, hi)
        total += int(bad.sum())
        df.loc[bad, c] = np.nan
    # температура процесса физически не может быть ниже температуры воздуха
    inv = df['process_temp_k'] < df['air_temp_k']
    total += int(inv.sum())
    df.loc[inv, 'process_temp_k'] = np.nan
    log.append(('Физически недопустимые значения -> пропуск', total, ', '.join(SENSORS)))
    return df

def drop_duplicates(df, log):
    n0 = len(df)
    df = df.drop_duplicates()
    full = n0 - len(df)
    n1 = len(df)
    df = df.drop_duplicates(subset='udi', keep='first')
    by_key = n1 - len(df)
    log.append(('Удаление полных дубликатов', full, 'все поля'))
    log.append(('Удаление дубликатов по ключу UDI', by_key, 'udi'))
    return df.sort_values('udi').reset_index(drop=True)

def handle_missing(df, log, stats):
    """Стратегии заполнения выбраны по результатам EDA (лаба 2) и сравнения
стратегий на внесённых пропусках (test_robustness.py)."""
    # 1. Записи без ключа или с пропуском обобщённой метки отказа не восстанавливаются:
    # метка - эталон для проверки базы правил, подставлять её нельзя.
    key_bad = df['udi'].isna() | df['product_id'].isna() | df['machine_failure'].isna()

```

```

# 2. Записи, где утрачено больше половины показаний датчиков, тоже удаляются.
sens_bad = df[SENSORS].isna().sum(axis=1) > len(SENSORS) // 2
drop = key_bad | sens_bad
log.append(('Удаление записей без ключа/метки или с > 50 % пропусков датчиков',
          int(drop.sum()), 'udi, product_id, machine_failure, датчики'))
df = df[~drop].copy()

filled = {}
# Класс качества - модой (категориальный признак)
m = df['quality_class'].isna()
if m.any():
    df.loc[m, 'quality_class'] = df['quality_class'].mode()[0]
filled['quality_class'] = int(m.sum())

# Износ инструмента: внутри цикла инструмента износ растёт с шагом,
# заданным классом предыдущего изделия (2/3/5 мин для L/M/H). Пропуск
# восстанавливается по предыдущему значению и шагу; если следующее известно
# значение меньше предыдущего (между ними была замена инструмента) -
# обратным счётом от следующего значения.
w = df['tool_wear_min'].to_numpy(dtype=float).copy()
step = df['quality_class'].astype(str).map(WEAR_STEP).to_numpy(dtype=float)
miss = np.where(np.isnan(w))[0]
for i in miss:
    if i == 0:
        w[i] = 0.0
        continue
    fwd = w[i - 1] + step[i - 1]
    nxt = next((j for j in range(i + 1, len(w)) if not np.isnan(w[j])), None)
    if nxt is not None and w[nxt] < w[i - 1]:
        w[i] = max(w[nxt] - step[i:nxt].sum(), 0.0)
    else:
        w[i] = fwd
filled['tool_wear_min'] = len(miss)
df['tool_wear_min'] = w

# Температуры генерируются как плавный временной ряд (случайное блуждание
# вдоль порядка записей), поэтому лучшая стратегия - линейная интерполяция
# по соседним записям (сравнение стратегий - в test_robustness.py).
m_air, m_proc = df['air_temp_k'].isna(), df['process_temp_k'].isna()
for c in ('air_temp_k', 'process_temp_k'):
    df[c] = df[c].interpolate(limit_direction='both').round(1)

# Момент: мощность привода примерно постоянна (корреляция с оборотами -0,875),
# поэтому момент = медианная мощность / угловая скорость.
# Обратный пересчёт оборотов по моменту не применяется: при малом моменте
# он даёт большую ошибку (см. отчёт, табл. сравнения стратегий).
m_t, m_n = df['torque_nm'].isna(), df['rot_speed_rpm'].isna()
omega = 2 * np.pi / 60
df.loc[m_t & ~m_n, 'torque_nm'] = (POWER_MED / (df['rot_speed_rpm'] * omega)).round(1)
# Остальное (обороты; пары с двумя пропусками) - медиана внутри класса качества
# (устойчива к выбросам, которых в моменте и оборотах много)
for c, m in [('air_temp_k', m_air), ('process_temp_k', m_proc),
             ('torque_nm', m_t), ('rot_speed_rpm', m_n)]:
    rest = df[c].isna()
    if rest.any():
        med = df.groupby('quality_class', observed=True)[c].transform('median')
        df.loc[rest, c] = med[rest]
    filled[c] = int(m.sum())

# Подтипы отказа: пропуск -> 0. Если при этом обобщённая метка = 1 и других
# подтипов нет, запись получит флаг failure_type_unknown (см. fix_labels).
for c in MODES + ['rnf']:
    m = df[c].isna()
    df.loc[m, c] = 0
    filled[c] = int(m.sum())

for c, n in filled.items():
    if n:
        log.append((f'Заполнение пропусков: {c}', n, c))
stats['filled'] = filled
return df

def iqr_bounds(s):
    q1, q3 = s.quantile(.25), s.quantile(.75)
    return q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)

def flag_outliers(df, log, stats):
    """Выбросы не удаляются: в лабе 2 показано, что это предотказные режимы."""

```

```

    out = {}
    for c, flag in [('rot_speed_rpm', 'is_outlier_speed'), ('torque_nm',
'is_outlier_torque')]:
        lo, hi = IQR_BOUNDS[c]
        # Значения точно на границе тоже помечаются: из 5 таких записей по моменту
        # (12,8 и 67,2 Н·м) 4 - отказы; так же их учитывал расчёт в лабе 2.
        mask = (df[c] <= lo) | (df[c] >= hi)
        df[flag] = mask.astype('int8')
        z = (df[c] - df[c].mean()).abs() / df[c].std()
        out[c] = {'lo': lo, 'hi': hi, 'n': int(mask.sum()), 'actual': iqr_bounds(df[c]),
                'fail': int((mask & (df['machine_failure'] == 1)).sum()),
                'z3': int((z > 3).sum())}
        log.append((f'Флаг выброса {flag} (IQR), записи сохранены', int(mask.sum()), c))
    stats['outliers'] = out
    return df

def fix_labels(df, log):
    sub = df[MODES].sum(axis=1)
    df['failure_type_unknown'] = ((df['machine_failure'] == 1) & (sub == 0)).astype('int8')
    df['n_failure_modes'] = sub.astype('int8')
    # Одноклассовая метка: подтип с наивысшим приоритетом по тяжести последствий
    mode = pd.Series('NONE', index=df.index)
    for c in reversed(PRIORITY):
        mode[df[c] == 1] = c.upper()
    mode[df['failure_type_unknown'] == 1] = 'UNKNOWN'
    df['failure_mode'] = pd.Categorical(mode, categories=['NONE', 'TWf', 'Hdf', 'Pwf', 'OSf',
'UNKNOWN'])
    # RNF исключается из целевых переменных и сохраняется как справочный признак
    rnf_inconsistent = int(((df['rnf'] == 1) & (df['machine_failure'] == 0)).sum())
    df = df.rename(columns={'rnf': 'rnf_ref'})
    log.append(('Флаг отказа без подтипа failure_type_unknown',
int(df['failure_type_unknown'].sum()), 'machine_failure'))
    log.append(('Записи с несколькими подтипами -> failure_mode по приоритету', int((sub >
1).sum()), 'twf, hdf, pwf, osf'))
    log.append(('RNF выведен из целевых переменных (rnf_ref); несогласованных меток',
rnf_inconsistent, 'rnf'))
    return df

def add_features(df, log):
    cls = df['quality_class'].astype(str)
    df['temp_diff_k'] = (df['process_temp_k'] - df['air_temp_k']).round(1)
    df['power_w'] = (df['torque_nm'] * df['rot_speed_rpm'] * 2 * np.pi / 60).round(1)
    df['wear_torque'] = (df['tool_wear_min'] * df['torque_nm']).round(1)
    df['osf_limit'] = cls.map(OSF_LIMIT).astype('int32')
    df['osf_margin'] = (df['osf_limit'] - df['wear_torque']).round(1)
    # запас по мощности до ближайшей границы допустимого диапазона, Вт (< 0 - вне диапазона)
    df['power_margin'] = np.minimum(df['power_w'] - POWER_MIN, POWER_MAX -
df['power_w']).round(1)
    # Цикл инструмента: износ сбрасывается в 0 при замене инструмента
    reset = df['tool_wear_min'].diff() < 0
    df['tool_cycle_id'] = (reset.cumsum() + 1).astype('int16')
    df['cycle_pos'] = (df.groupby('tool_cycle_id').cumcount() + 1).astype('int16')
    new = ['temp_diff_k', 'power_w', 'wear_torque', 'osf_limit', 'osf_margin',
        'power_margin', 'tool_cycle_id', 'cycle_pos']
    log.append(('Расчёт производных признаков', len(new), ', '.join(new)))
    return df

def downcast(df):
    for c in LABELS[:-1] + ['rnf_ref']:
        df[c] = df[c].astype('int8')
    df['udi'] = df['udi'].astype('int32')
    df['rot_speed_rpm'] = df['rot_speed_rpm'].round().astype('int32')
    df['tool_wear_min'] = df['tool_wear_min'].astype('int16')
    return df

ORDER = ['udi', 'product_id', 'product_serial', 'quality_class',
        'air_temp_k', 'process_temp_k', 'rot_speed_rpm', 'torque_nm', 'tool_wear_min',
        'temp_diff_k', 'power_w', 'power_margin', 'wear_torque', 'osf_limit', 'osf_margin',
        'tool_cycle_id', 'cycle_pos', 'is_outlier_speed', 'is_outlier_torque',
        'machine_failure', 'twf', 'hdf', 'pwf', 'osf', 'failure_mode', 'n_failure_modes',
        'failure_type_unknown', 'rnf_ref']

def transform(raw):
    log, stats = [], {}
    df = rename(raw.copy(), log)

```

```

df = normalize_text(df, log)
df = cast_types(df, log)
df = check_ranges(df, log)
df = drop_duplicates(df, log)
df = handle_missing(df, log, stats)
df = flag_outliers(df, log, stats)
df = fix_labels(df, log)
df = add_features(df, log)
df = downcast(df)
return df[ORDER].reset_index(drop=True), log, stats

# --- валидация -----
def validate(df):
    """Утверждения о качестве результата; при нарушении конвейер останавливается."""
    checks = {
        'нет пропусков': int(df.isnull().sum().sum()) == 0,
        'UDI уникален': df['udi'].is_unique,
        'Product ID уникален': df['product_id'].is_unique,
        'класс совпадает с буквой Product ID': bool((df['product_id'].str[0] ==
df['quality_class'].astype(str)).all()),
        'показания в допустимых диапазонах': all(df[c].between(*VALID_RANGE[c]).all() for c in
SENSORS),
        'метки бинарные': bool(df[LABELS[:-1] + ['rnf_ref']].isin([0, 1]).all().all()),
        'machine_failure = OR(подтипов) или failure_type_unknown':
            bool(((df[MODES].max(axis=1) | df['failure_type_unknown']) ==
df['machine_failure']).all()),
        'объём не меньше 500 записей': len(df) >= 500,
    }
    return checks

def wear_step_consistency(df):
    """Доля шагов износа, совпадающих с нормативом по классу предыдущего изделия."""
    d = df['tool_wear_min'].diff()
    prev = df['quality_class'].astype(str).shift(1).map(WEAR_STEP)
    m = d >= 0
    return int((d[m] == prev[m]).sum()), int(m.sum())

# --- L: сохранение -----
def quality_report(df):
    return pd.DataFrame({
        'Поле': df.columns,
        'Тип': df.dtypes.astype(str).values,
        'Уникальных': df.nunique().values,
        'Пропусков': df.isnull().sum().values,
        'Пропусков_%': (df.isnull().sum() / len(df) * 100).round(2).values,
        'Мин': [df[c].min() if df[c].dtype.kind in 'if' else '' for c in df.columns],
        'Макс': [df[c].max() if df[c].dtype.kind in 'if' else '' for c in df.columns],
    })

def load(df, log, out=OUT):
    out.mkdir(exist_ok=True)
    df.to_csv(out / 'data_cleaned.csv', index=False)
    df.to_parquet(out / 'data_cleaned.parquet', index=False)
    quality_report(df).to_csv(out / 'quality_report.csv', index=False)
    pd.DataFrame(log, columns=['Операция', 'Затраты', 'Поля']).to_csv(out / 'etl_log.csv',
index=False)

def run(raw_path=RAW, out=OUT, save=True):
    raw = extract(raw_path)
    df, log, stats = transform(raw)
    checks = validate(df)
    failed = [k for k, ok in checks.items() if not ok]
    if failed:
        raise AssertionError(f'Валидация не пройдена: {failed}')
    if save:
        load(df, log, out)
    stats.update({
        'source_md5': md5(raw_path),
        'rows_in': len(raw), 'rows_out': len(df),
        'cols_in': raw.shape[1], 'cols_out': df.shape[1],
        'mem_in_kb': raw.memory_usage(deep=True).sum() / 1024,
        'mem_out_kb': df.memory_usage(deep=True).sum() / 1024,
        'checks': checks,
        'wear_step': wear_step_consistency(df),
    })

```

```

    return raw, df, log, stats

if __name__ == '__main__':
    raw, df, log, stats = run()
    print(f'Исходный размер: {raw.shape}, итоговый: {df.shape}')
    for op, n, cols in log:
        print(f' {n:>6} {op}')
    print('Проверки:', 'все пройдены' if all(stats['checks'].values()) else stats['checks'])
    print(f"Память: {stats['mem_in_kb']:.0f} КБ -> {stats['mem_out_kb']:.0f} КБ")
    json.dump(stats, open(OUT / 'etl_stats.json', 'w'), ensure_ascii=False, indent=1,
default=float)
    sys.exit(0)

```